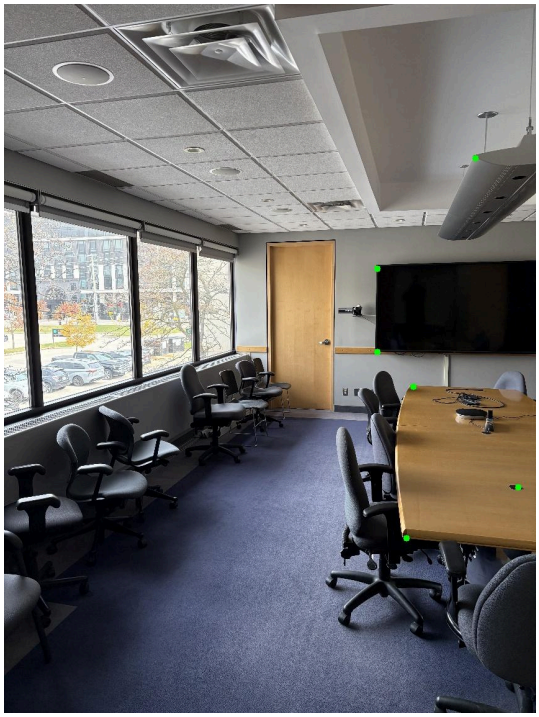
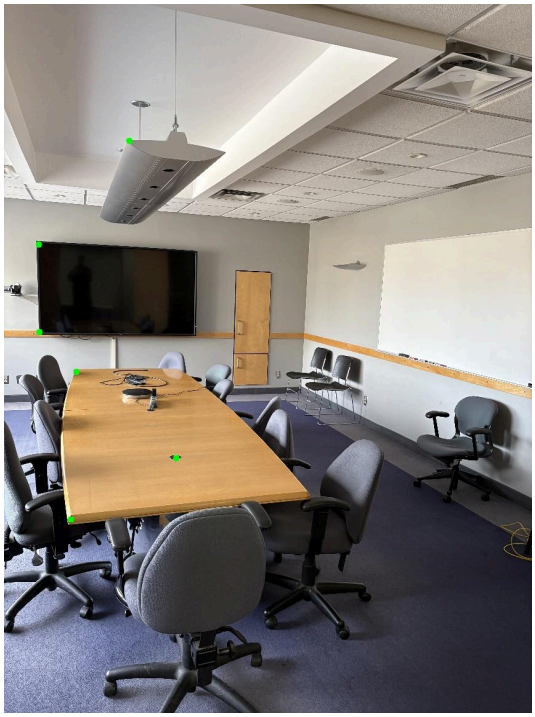
SYDE671 Assignment 3 - Arthur Kowara
https://akowara.github.io/SYDE671/3/index.html https://github.com/akowara/SYDE671/tree/main

Part A:
2.
The following images show the shared points selected to calculate the homography between two images. The same was tried using only 4 points, but that led to poor warping, and that is why six points were selected. When selecting the points, there was a focus on selecting sharp features in order, such as corners, which helps ensure shared features were selected and in the correct order.



The following shows the points correspondences from each image, the system of equations and the recovered homography matrix. The homography matrix computed with OpenCV matches

the H matrix computed using the code for part A.2. The homography matrices are quite similar so that is a good sign.

Point correspondences (image 1 → image 2)

#	x	y	u	v
1	602	422	56	386
2	600	562	61	532
3	657	618	122	586
4	643	857	109	839
5	826	782	279	730
6	758	247	203	221

System $A\mathbf{h} = \mathbf{0}$ (two rows per correspondence)

$$\underbrace{\begin{bmatrix} -602 & -422 & -1 & 0 & 0 & 0 & 33712 & 23632 & 56 \\ 0 & 0 & 0 & -602 & -422 & -1 & 232372 & 162892 & 386 \\ -600 & -562 & -1 & 0 & 0 & 0 & 36600 & 34282 & 61 \\ 0 & 0 & 0 & -600 & -562 & -1 & 319200 & 298984 & 532 \\ -657 & -618 & -1 & 0 & 0 & 0 & 80154 & 75396 & 122 \\ 0 & 0 & 0 & -657 & -618 & -1 & 385002 & 362148 & 586 \\ -643 & -857 & -1 & 0 & 0 & 0 & 70087 & 93413 & 109 \\ 0 & 0 & 0 & -643 & -857 & -1 & 539477 & 719023 & 839 \\ -826 & -782 & -1 & 0 & 0 & 0 & 230454 & 218178 & 279 \\ 0 & 0 & 0 & -826 & -782 & -1 & 602980 & 570860 & 730 \\ -758 & -247 & -1 & 0 & 0 & 0 & 153874 & 50141 & 203 \\ 0 & 0 & 0 & -758 & -247 & -1 & 167518 & 54587 & 221 \end{bmatrix}}_{A\ (12\times 9)} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \\ h_{33} \end{bmatrix} = \mathbf{0}$$

Solution (SVD, normalized so $h_{33} = 1$)

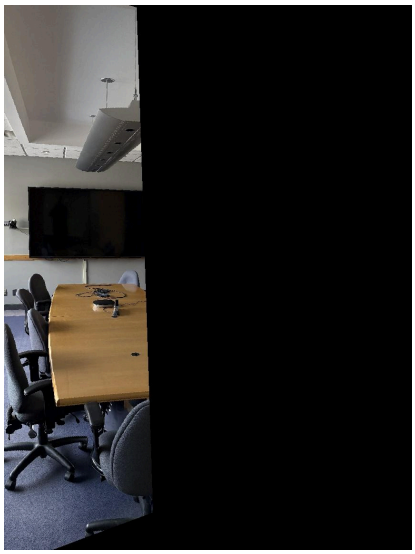
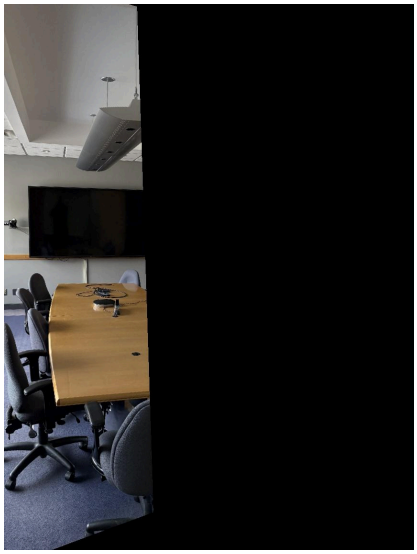
$$H_{a_2} = \begin{bmatrix} 1.5064 & 0.0287 & -837.55 \\ 0.3016 & 1.4277 & -242.91 \\ 7.240 \times 10^{-4} & -7.662 \times 10^{-5} & 1 \end{bmatrix} \qquad H_{\text{OpenCV}} = \begin{bmatrix} 1.4819 & 0.0271 & -822.89 \\ 0.2869 & 1.4049 & -230.85 \\ 6.981 \times 10^{-4} & -8.058 \times 10^{-5} & 1 \end{bmatrix}$$

3.

Original Image Pair One:



Warped Image Pair One (Nearest Neighbour) (Blinear):

Speeds:

- Nearest Neighbour: 0.0486 s
- Bilinear: 0.0814s

Errors (Compared to OpenCV):

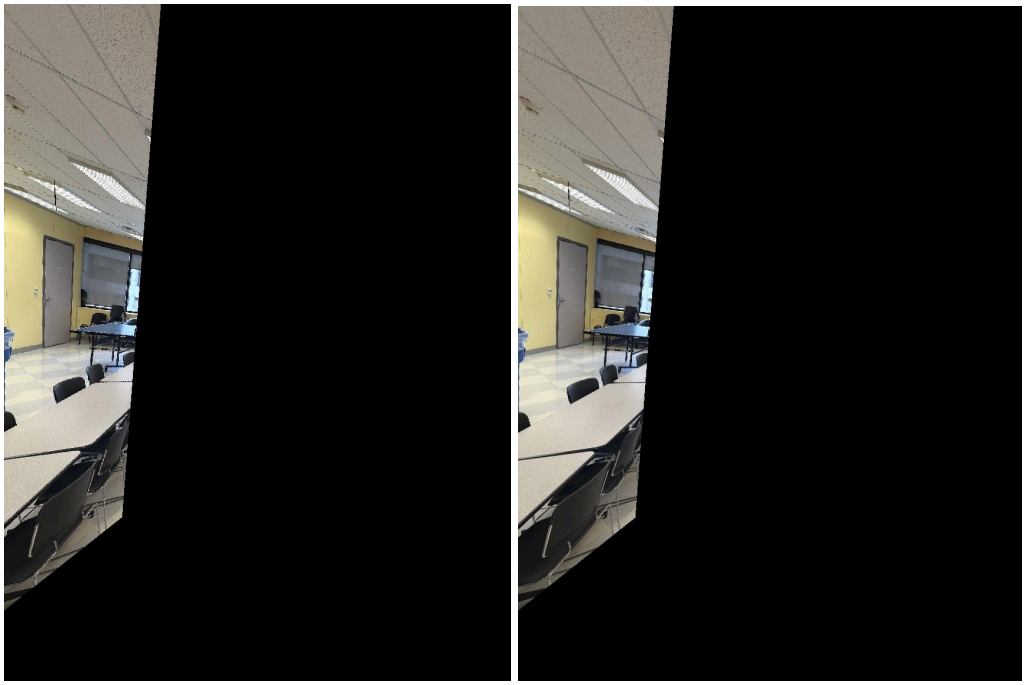
- Nearest Neighbour: 1.87e-05
- Bilinear: 0.001

Original Image Pair Two:





Warped Image Pair Two (Nearest Neighbour) (Blinear):



Speeds:

- Nearest Neighbour: 0.0564 s
- Bilinear: 0.0652 s

Errors (Compared to OpenCV):

- Nearest Neighbour1.73e-05
- Bilinear: 0.001

Original Image Pair Three:



Warped Image Pair Two (Nearest Neighbour) (Blinear):



Speeds:

- Nearest Neighbour: 0.0026 s
- Bilinear: 0.001 s

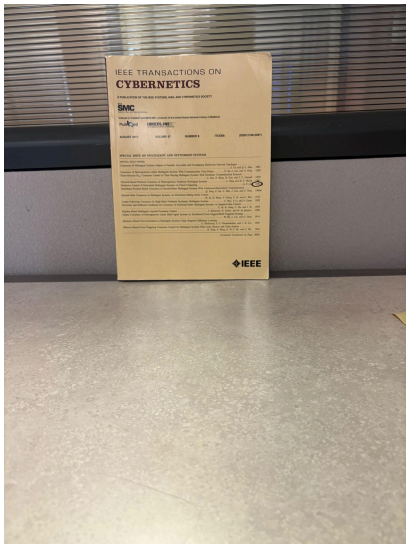
Errors (Compared to OpenCV):

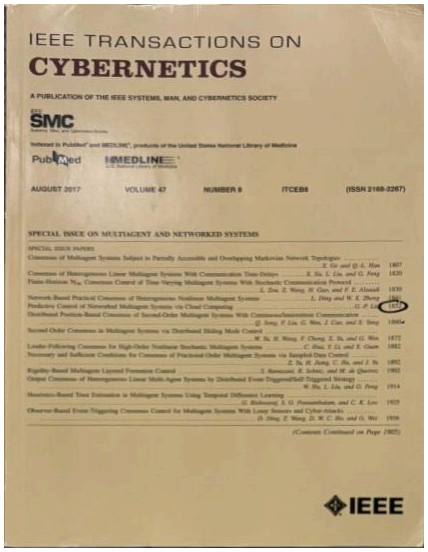
- Nearest Neighbour1.43e-05
- Bilinear: 0.001

Visually looking at the warps both interpolation methods produced, both outputs look similar. However, when zooming in and looking at the finer details, the bilinear method produces smoother edges, which is expected due to using a weighted average of nearby pixels instead of only one pixel.

Looking at the timing of all image pairs, the nearest neighbour algorithm performs quicker. This is expected, as the bilinear method utilizes four times as many memory reads as the nearest neighbour method. Given the size of the images used, it does not affect the program much. Both methods scale linearly with the size of the images used, so the relative cost stays about the same regardless of the image size. For this reason, the bilinear method has a better trade-off between quality and time compared to the nearest neighbour method.

Rectification:









The results of the rectification look good, suggesting that so far the homography matrix computation and warping functions are correctly working.

4.
Original Images


Coca-Cola



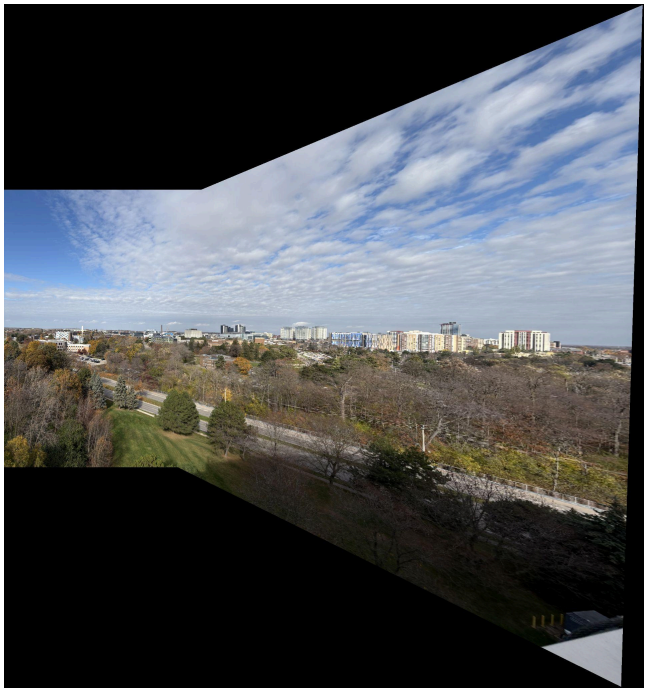




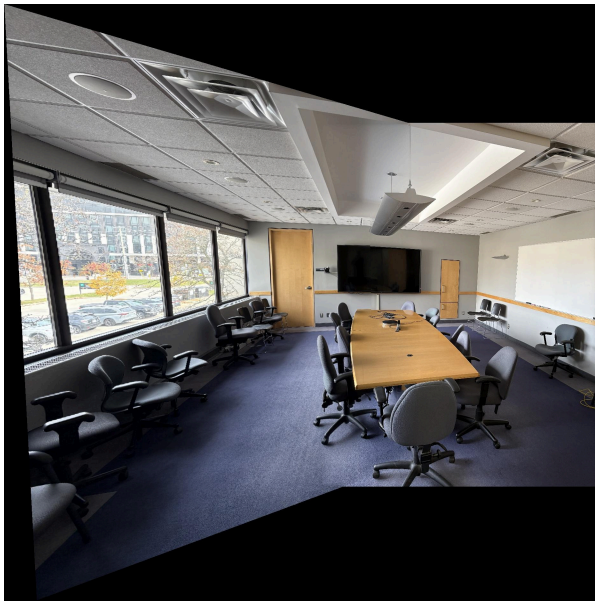


Bilinear Feather Blend

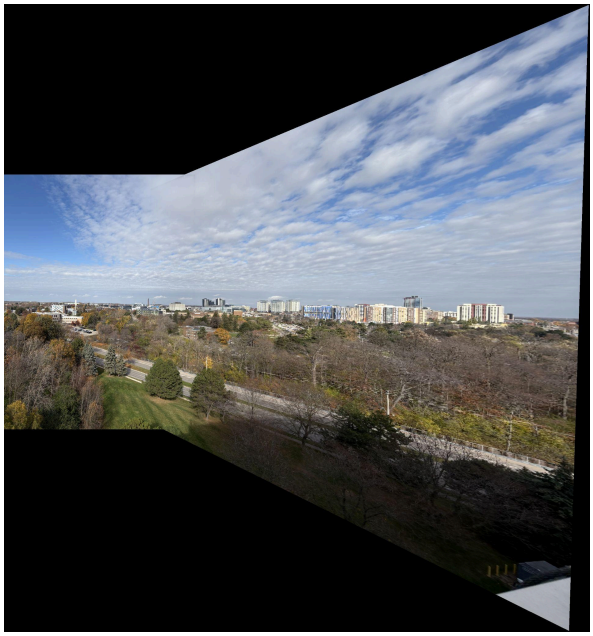




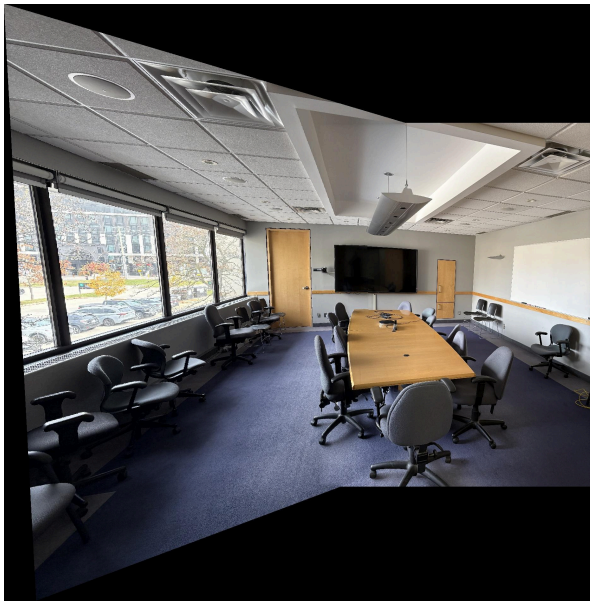
Bilinear Laplacian



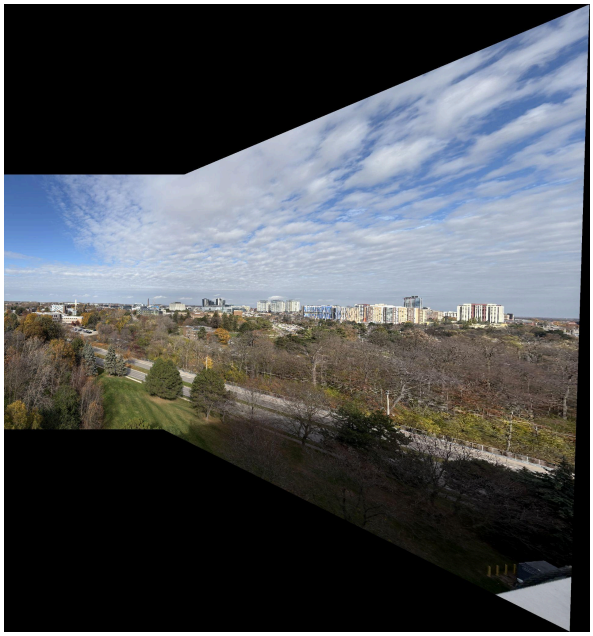




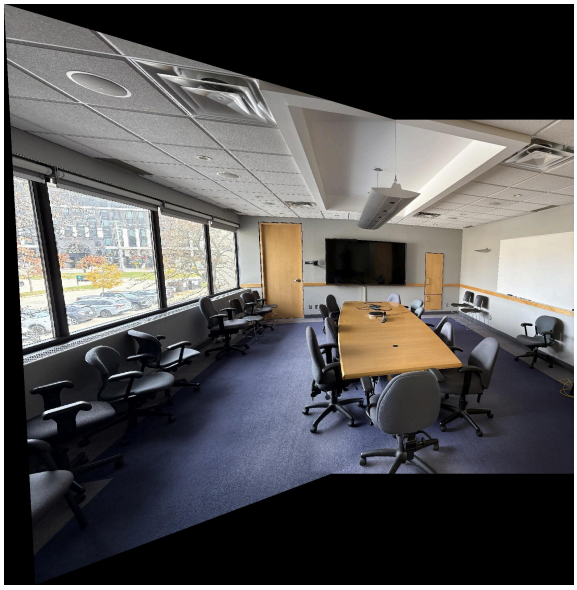
Nearest Neighbour Feather Blend



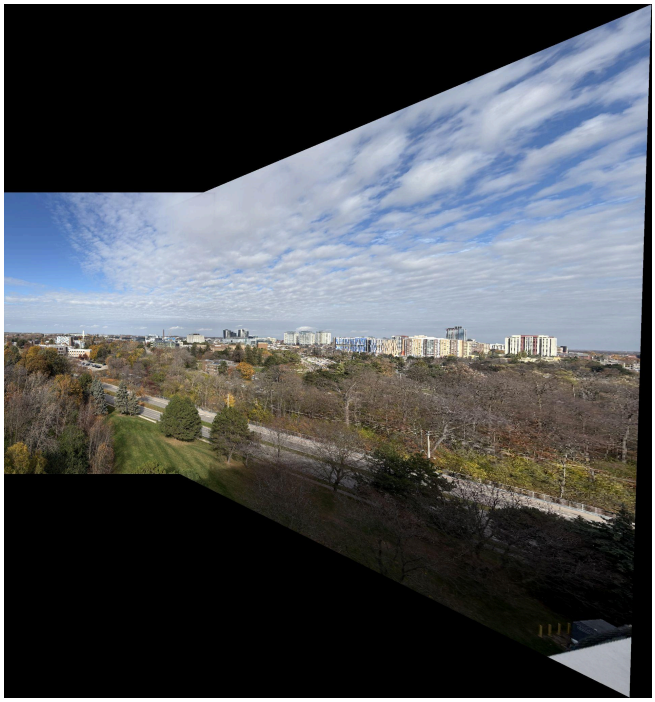




Nearest Neighbour Laplacian







In the previous section, the conclusion was that bilinear was the better option due to its trade-off between speed and quality, so the focus will be on using the bilinear interpolation method.

Process:

- Take two images of the same scene with decent overlap, with enough shared keypoints between the two.
- Select six key points that are shared between images. When selecting the key points there is a focus on selecting defined features in an intuitive order as this helps with accuracy and consistency. If the key points are selected in the wrong order, mosaics will look incorrect. If dull features are selected, it could lead to a mosaic that does not line up properly due to error between the points in the two images. This step is about calculating the homography matrix.
- When warping the image, the final mosaic canvas needs to be calculated. This is done by going through each image and its related H matrix in order to get the warped corners. Once all warped corners are calculated, it is easy to get the maximum and minimum x and y values to get the shape of the final mosaic. Along with this step is the calculation of the translation matrix which moves the bounding box of the mosaic to be lined up with the top left corner.
- Now that the final mosaic canvas is calculated, each image needs to be placed onto the canvas. In order to do this, the images need to be warped using the methods discussed in the previous question and a matching weight map needs

to be calculated. The weight map spans the mosaic and uses a distance transform to make the center of the image 1 while falling smoothly toward 0 at the edges. The mask is warped like the image so it lines up with the warped pixels.

- Now that the image is warped correctly, and the weight mask is calculated, the next step is to blend the images correctly using these values to create the mosaic. Two blending techniques were used as part of a one-shot procedure: feather blending and Laplacian pyramid blending.
- **Feather Blending:** Each output pixel is a weighted average of the overlapping images. The highest weight is one in the middle and fades to zero at the edges of the image, causing one image to take over the other gradually instead of abruptly switching between the two. This method tends to hide the seams between images better but averages fine details, which can show up as blur or ghosting.
- **Laplacian Pyramid Blending:** Each image is split into a low-frequency part and a detail part, which creates a Laplacian pyramid with a single level. This should improve on the blending process because different frequencies need different treatment. Each frequency is blended its own way and the results are added back together. Low frequencies should blend over a wider area so there is no visible seam, while high frequencies should blend over a narrow area. This is where Laplacian pyramid blending gets its advantages.

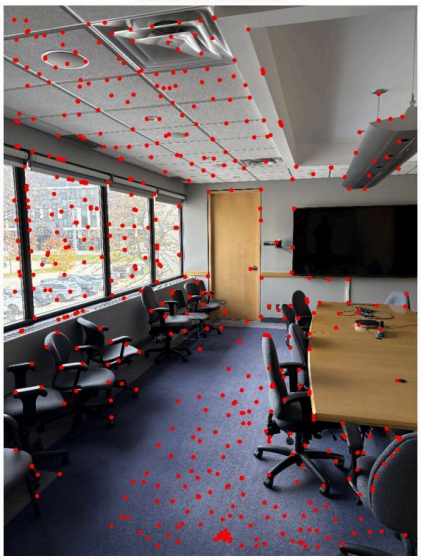
Comparing the two blending techniques, some ghosting and blur shows up when using feather blending. The Laplacian blending technique seems to improve these shortcomings, but the seam between the two images is more noticeable. Personally, the Laplacian blending technique creates a more realistic image, as it removes the ghosting at the cost of a more noticeable seam.

Part B:
1.

25822 corners before ANMS



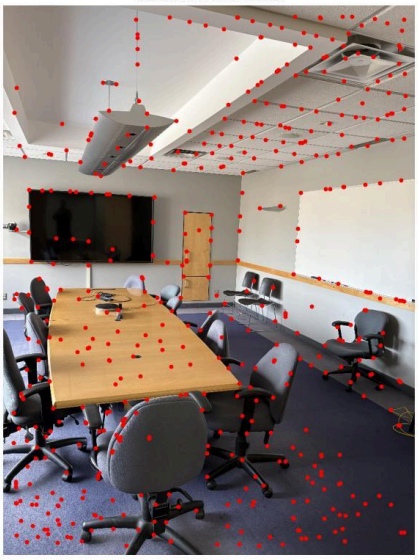
500 corners after ANMS



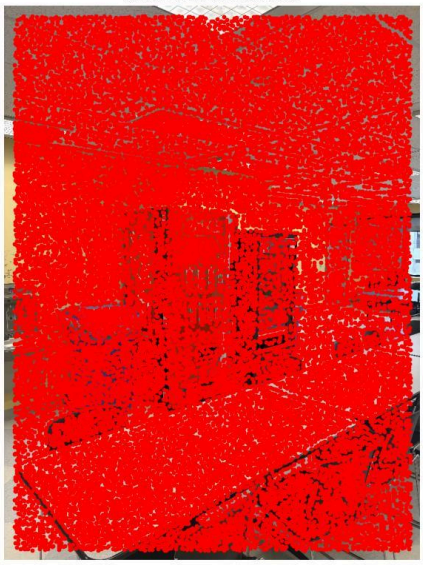
25670 corners before ANMS



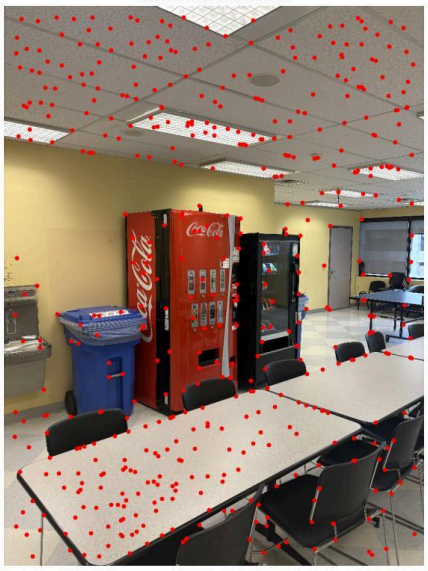
500 corners after ANMS



23953 corners before ANMS



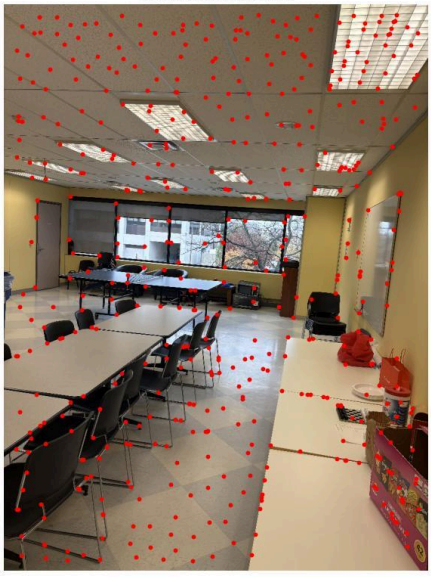
500 corners after ANMS

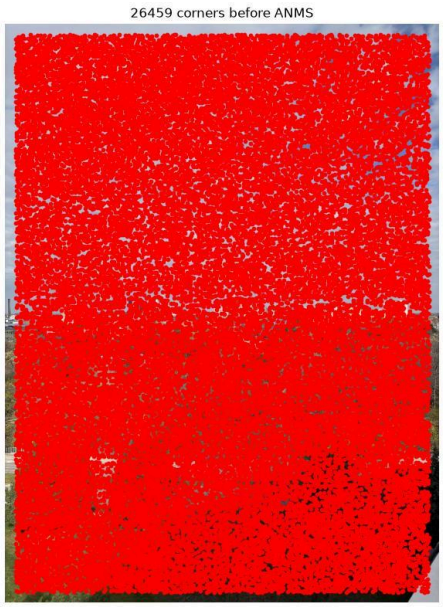


22879 corners before ANMS

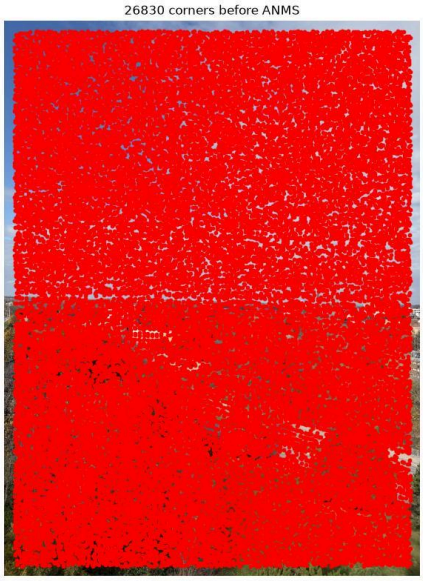


500 corners after ANMS







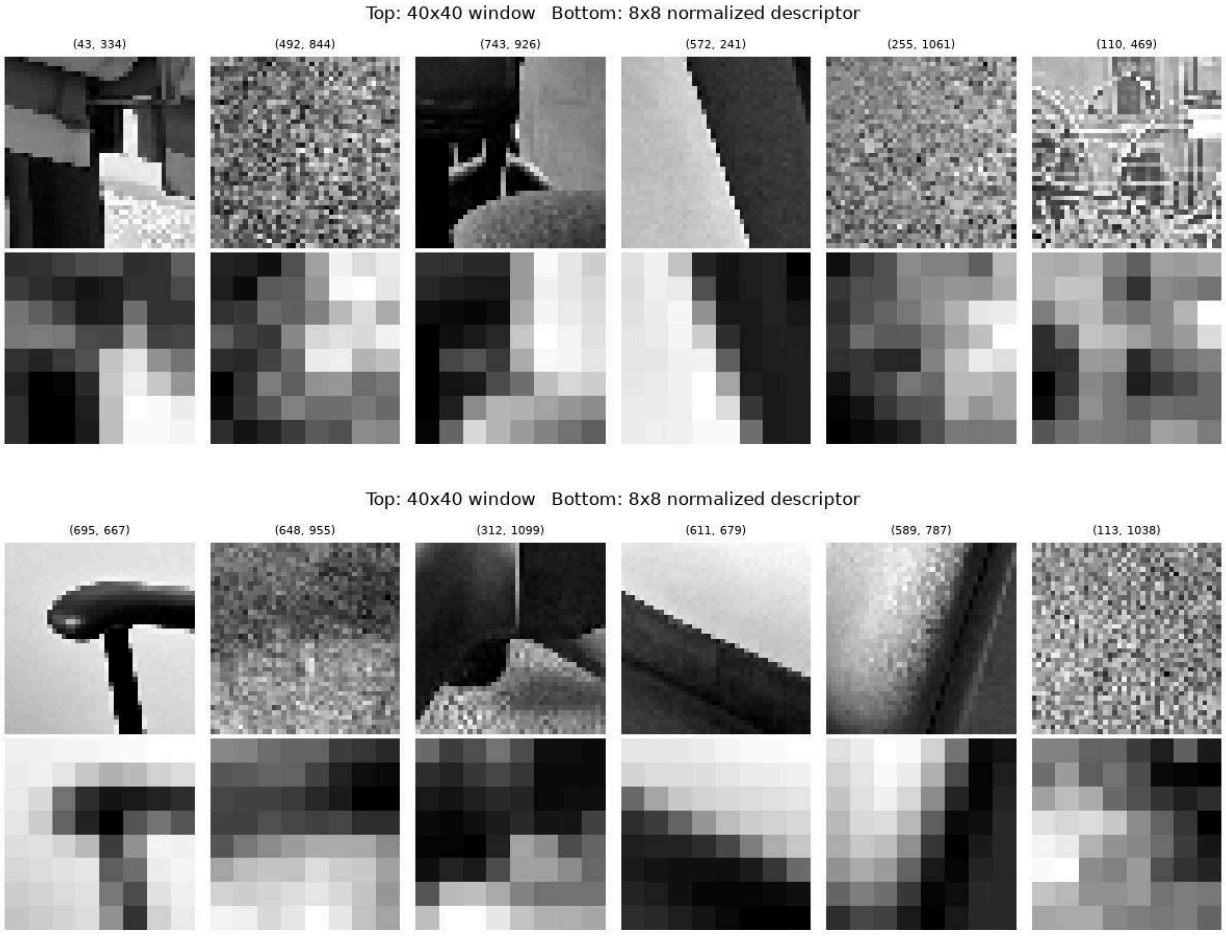




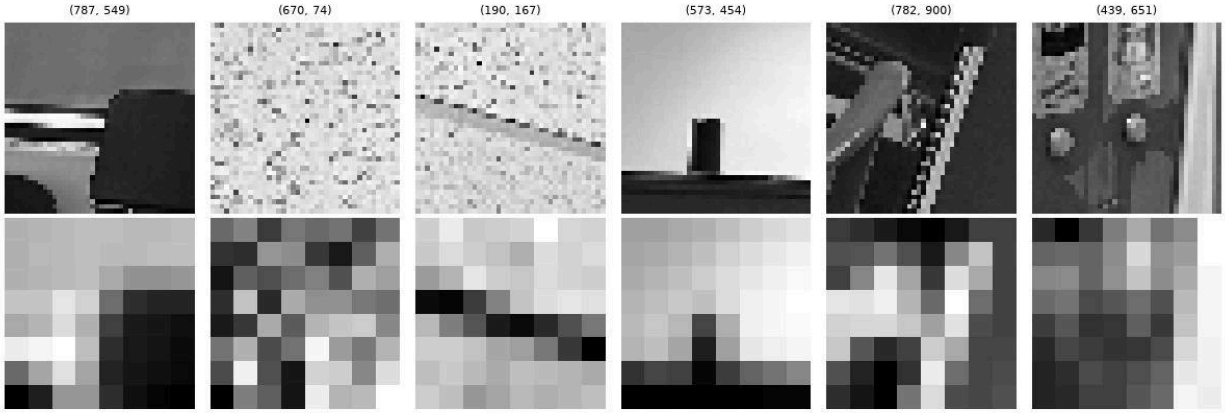
From the output images, it is clear that ANMS is a vital part of corner detection. The Harris corner detector is able to find a lot of corners, so many that they are not valuable until most of them are removed using a technique such as ANMS. The corner detection is the first step in the auto mosaic pipeline.

ANMS keeps the 500 corners with the largest suppression radius, where a corner's radius is the distance to the nearest corner that is significantly stronger than it (using a robustness constant of c = 0.9). This keeps strong corners while spreading them evenly across the image.

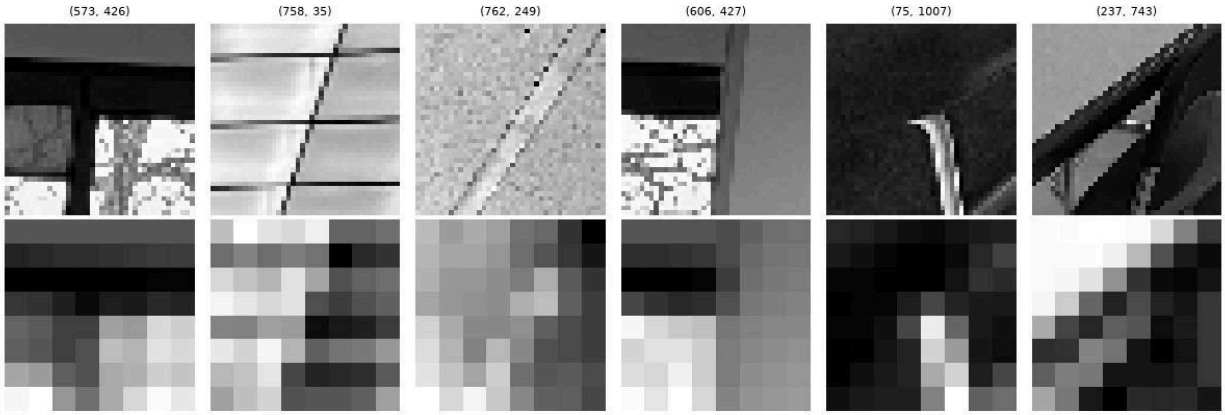
2.

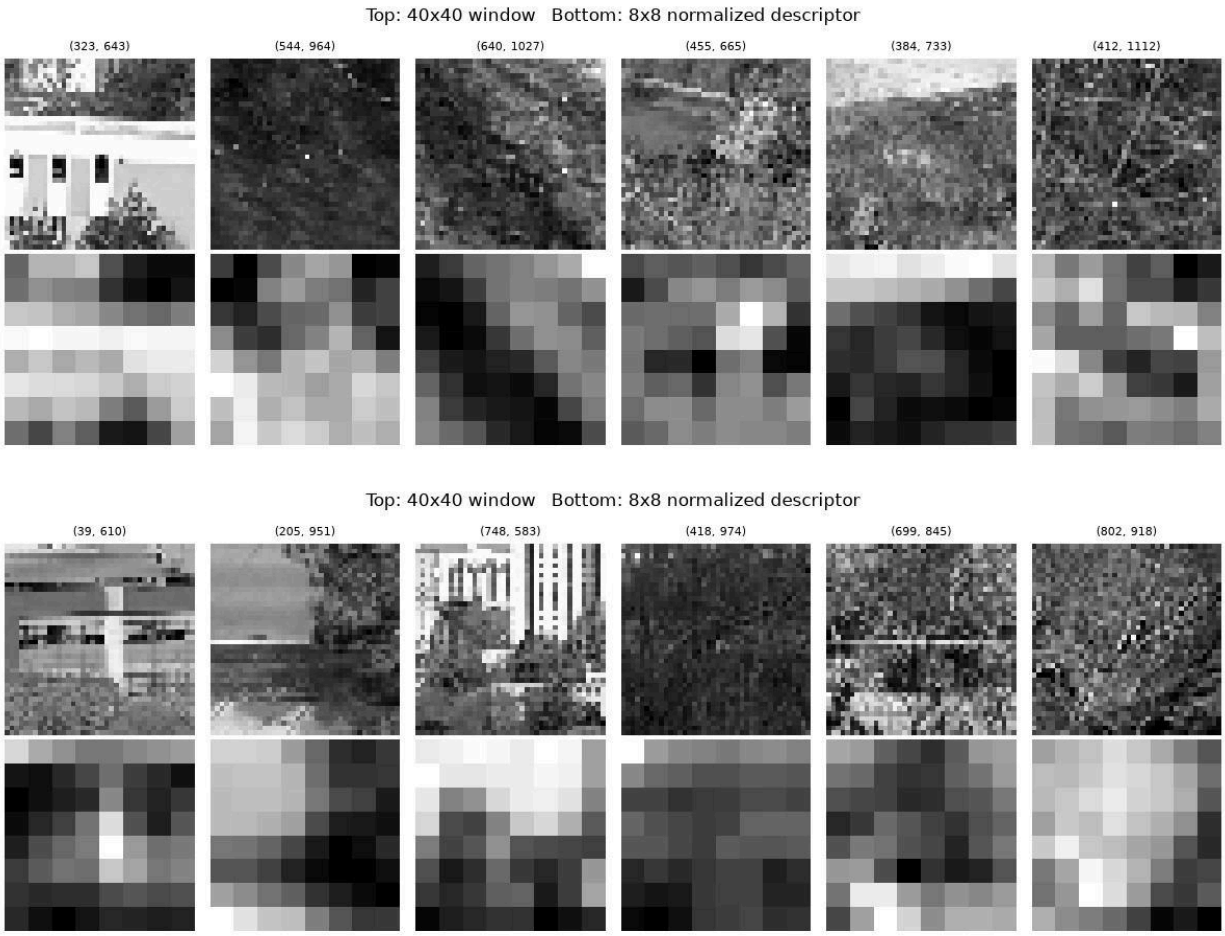

Top: 40x40 window Bottom: 8x8 normalized descriptor
(43, 334)
(492, 844)
(743, 926)
(572, 241)
(255, 1061)
(110, 469)
Top: 40x40 window Bottom: 8x8 normalized descriptor
(695, 667)
(648, 955)
(312, 1099)
(611, 679)
(589, 787)
(113, 1038)

Top: 40x40 window Bottom: 8x8 normalized descriptor



Top: 40x40 window Bottom: 8x8 normalized descriptor



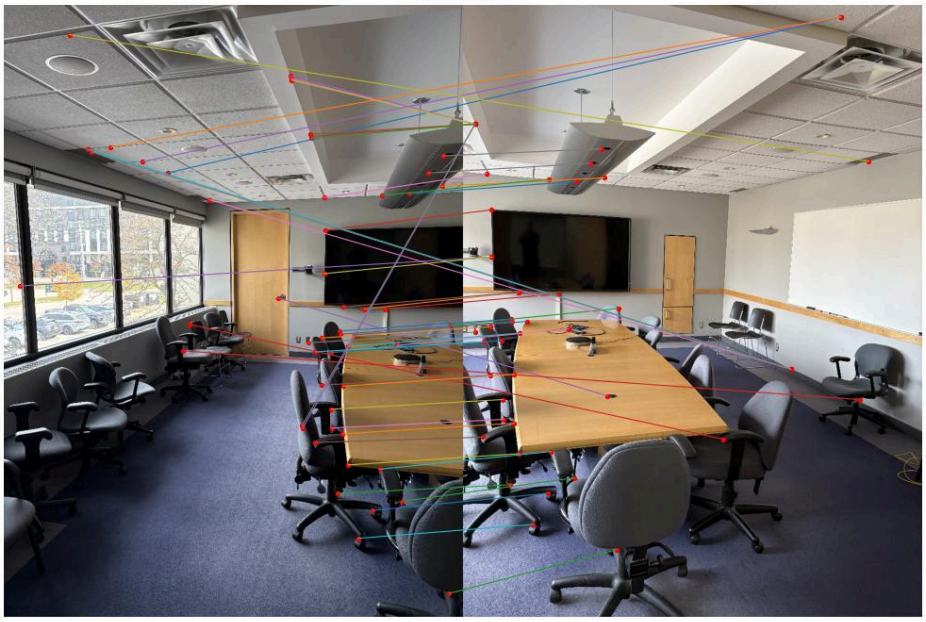


The images show some of the interest points which the Harris algorithm detects and how they are saved as feature descriptors. These descriptors will be used to find the same points in the other image. If an area in the other image resembles one of these descriptors, then it will think it is the same interest point; this will lead to feature matching in the next section.

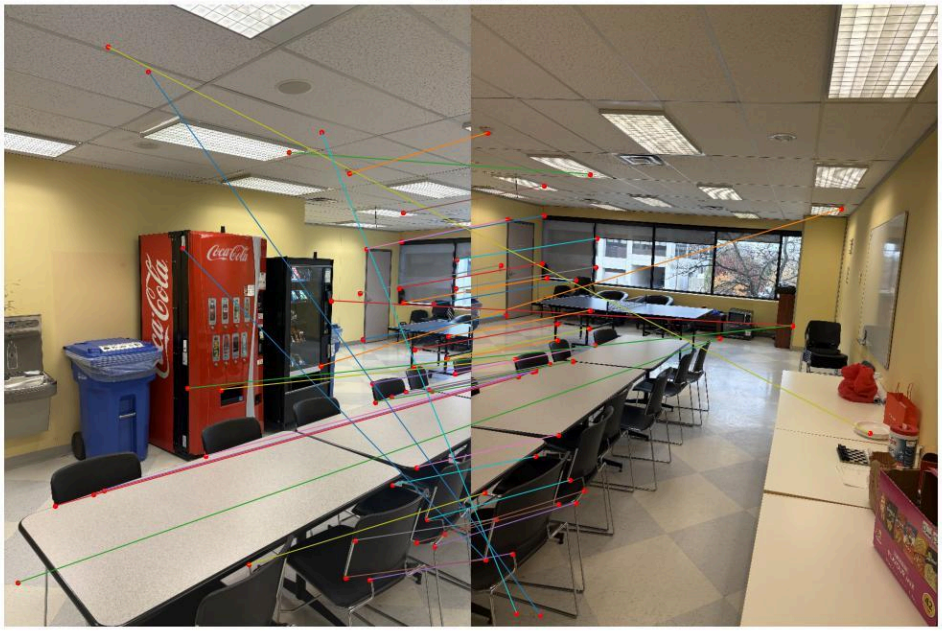
Each descriptor is created by blurring the image with a Gaussian ($\sigma = 2$), taking the 40x40 window around each corner and downsampling it to an 8x8 patch. Each patch is then bias/gain normalized by subtracting its mean and dividing by its standard deviation, which makes the descriptors less sensitive to changes in brightness and contrast.

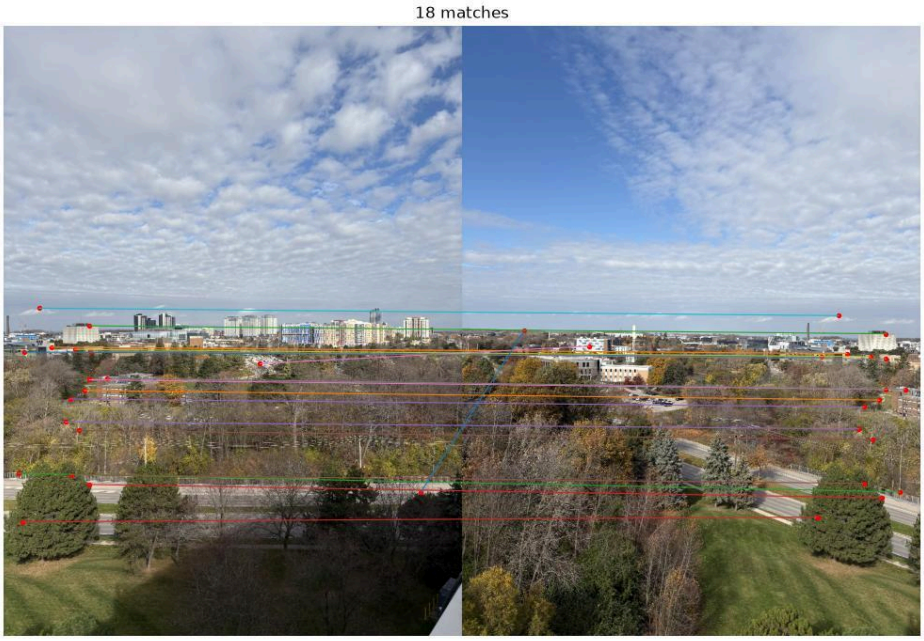
3.

53 matches



46 matches



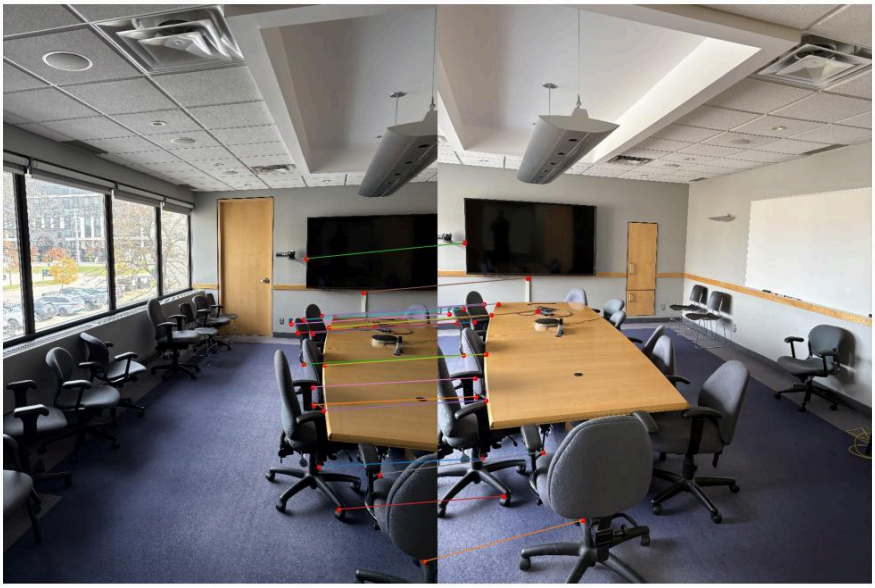


Overall, the feature matching does a pretty good job matching corresponding features between the two images; however, there are a few features that get matched that are wrong for each image. This is fine because RANSAC should filter out the bad matches during its homography computation process.

For each descriptor, the two nearest neighbours in the other image are found. Following Lowe's approach, a match is only kept if the ratio between the first and second nearest neighbour (squared) distances is below 0.5. This removes ambiguous matches, where a feature looks similar to more than one point in the other image.

4. RANSAC was run for 2000 iterations. In each iteration, four random matches are chosen and used to compute a homography, which is then applied to all of the matched points. A match counts as an inlier if its projected point lands within 2 pixels of its matching point. The homography with the most inliers is kept, and the final homography is recomputed using least squares on all of its inliers.

19 matches



Auto mosaic（images 0 & 1）: Nearest Neighbor + Feather Blend



Auto mosaic（images 0 & 1）: Bilinear + Feather Blend



Auto mosaic (images 0 & 1): Nearest Neighbor + Laplacian Blend

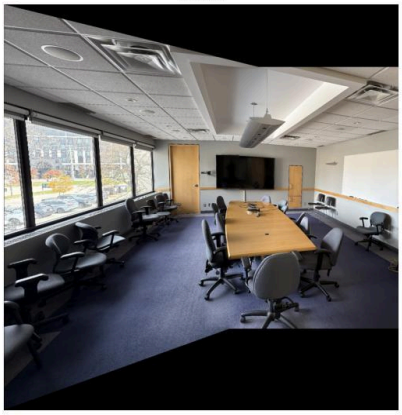


Auto mosaic（images 0 & 1）: Bilinear + Laplacian Blend

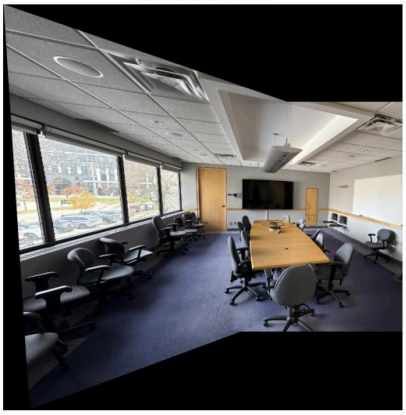


Manual vs Automatic Mosaic (images 0 & 1), Bilinear + Laplacian Blend

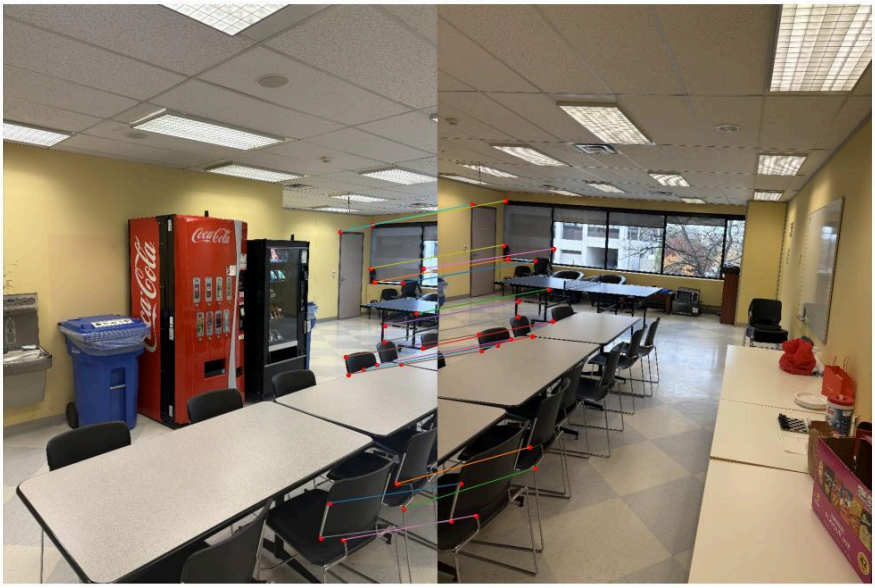
Manual



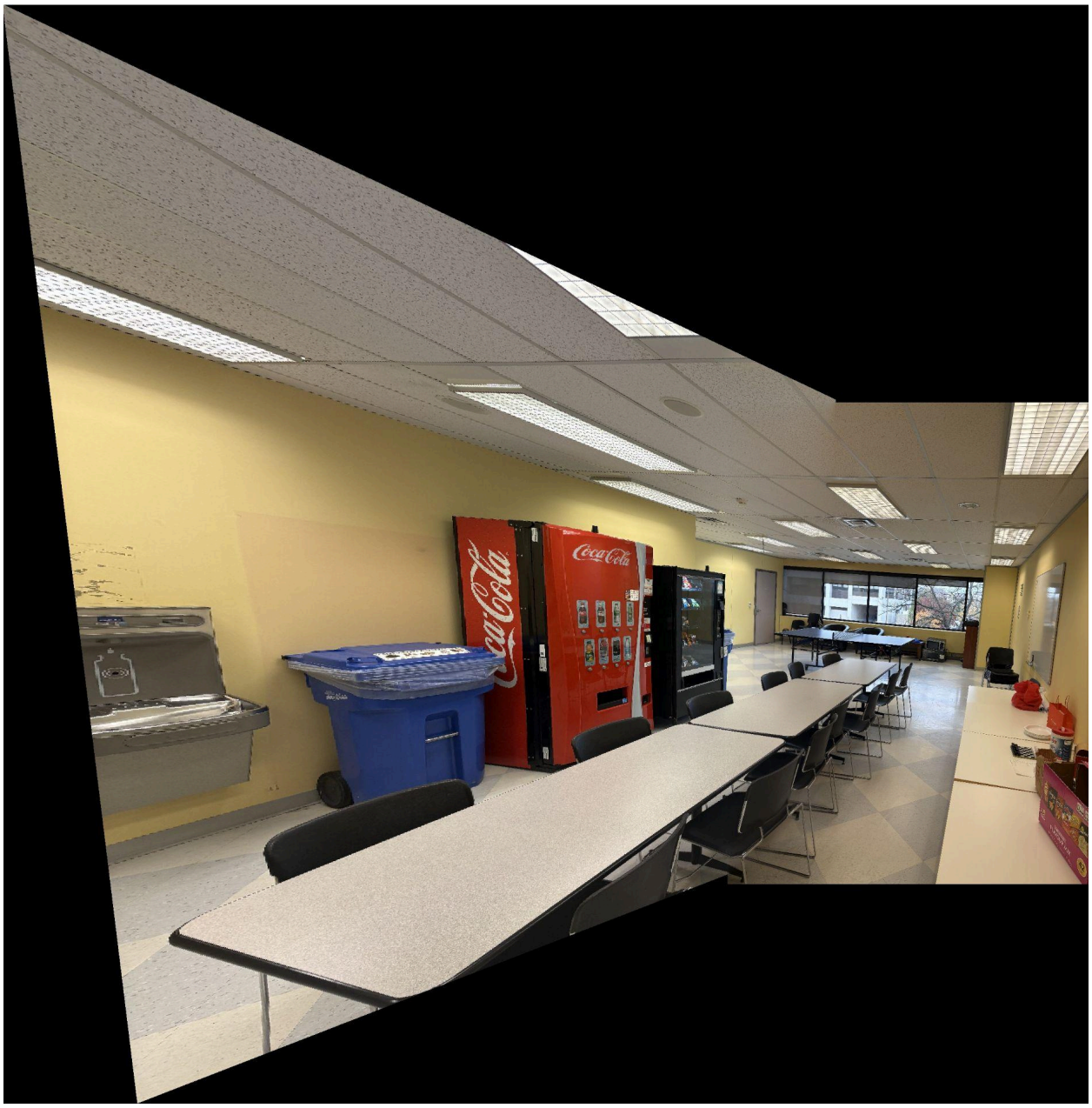
Automatic (RANSAC)



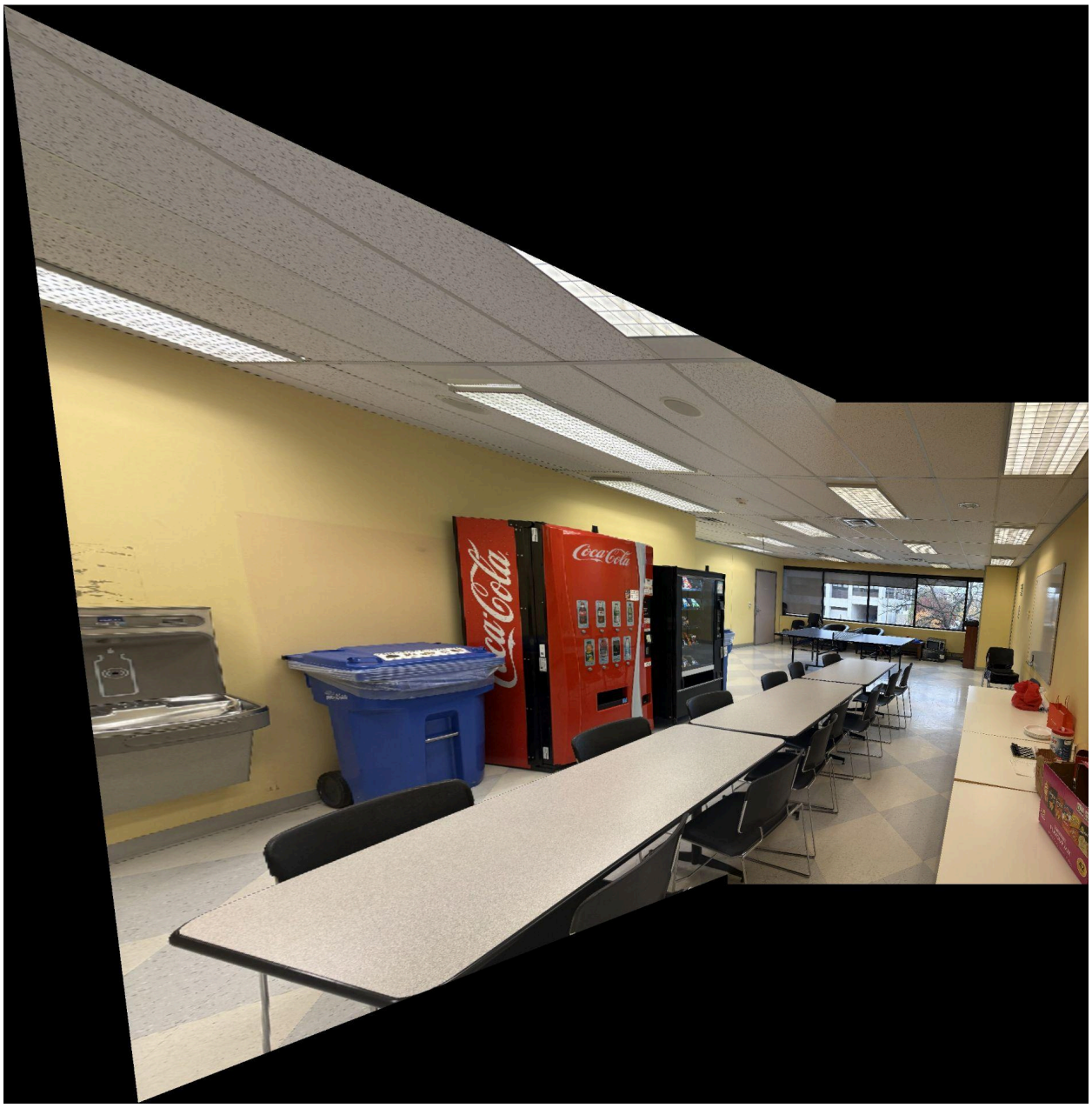
20 matches



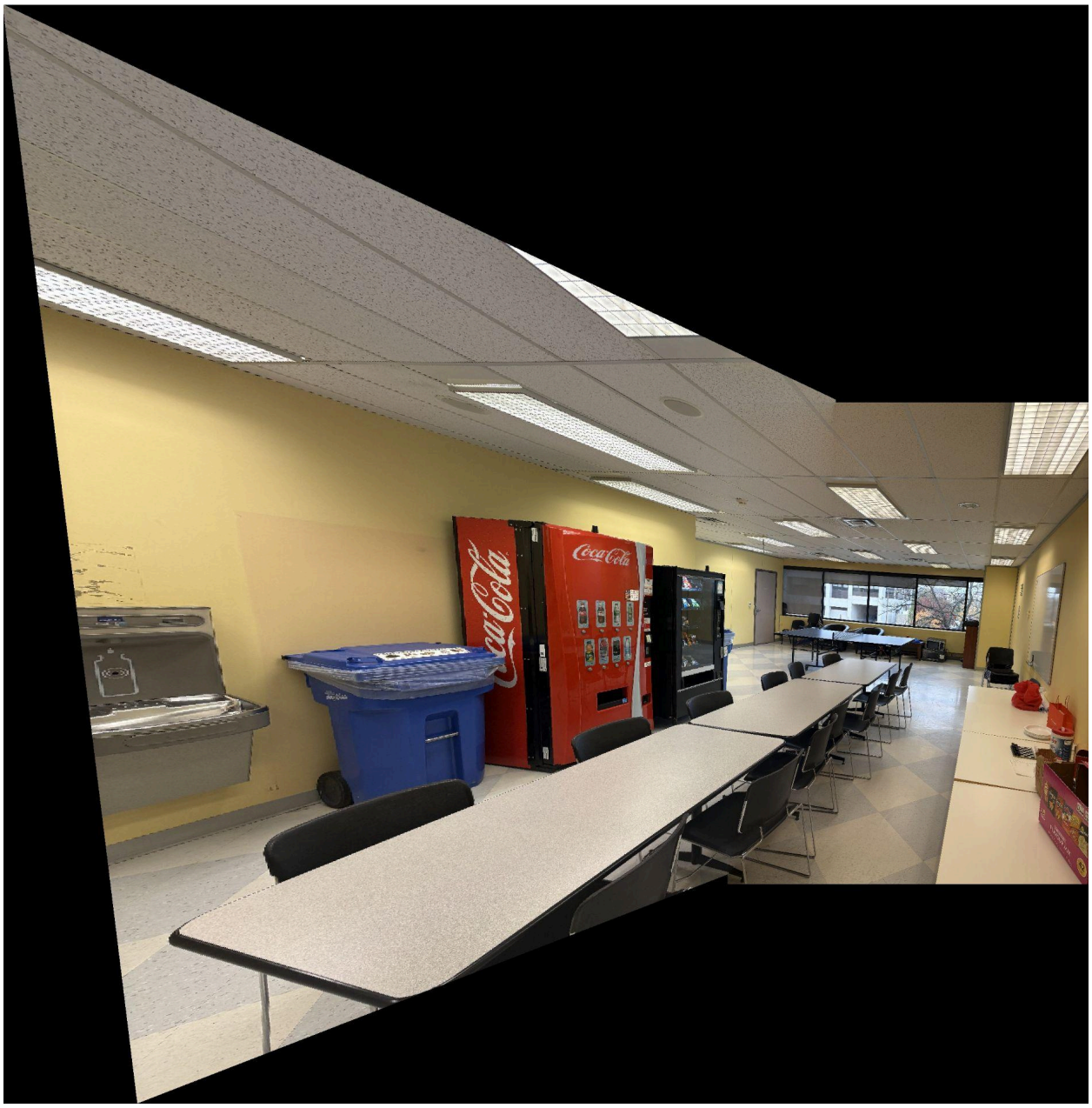
Auto mosaic（images 2 & 3）: Nearest Neighbor + Feather Blend



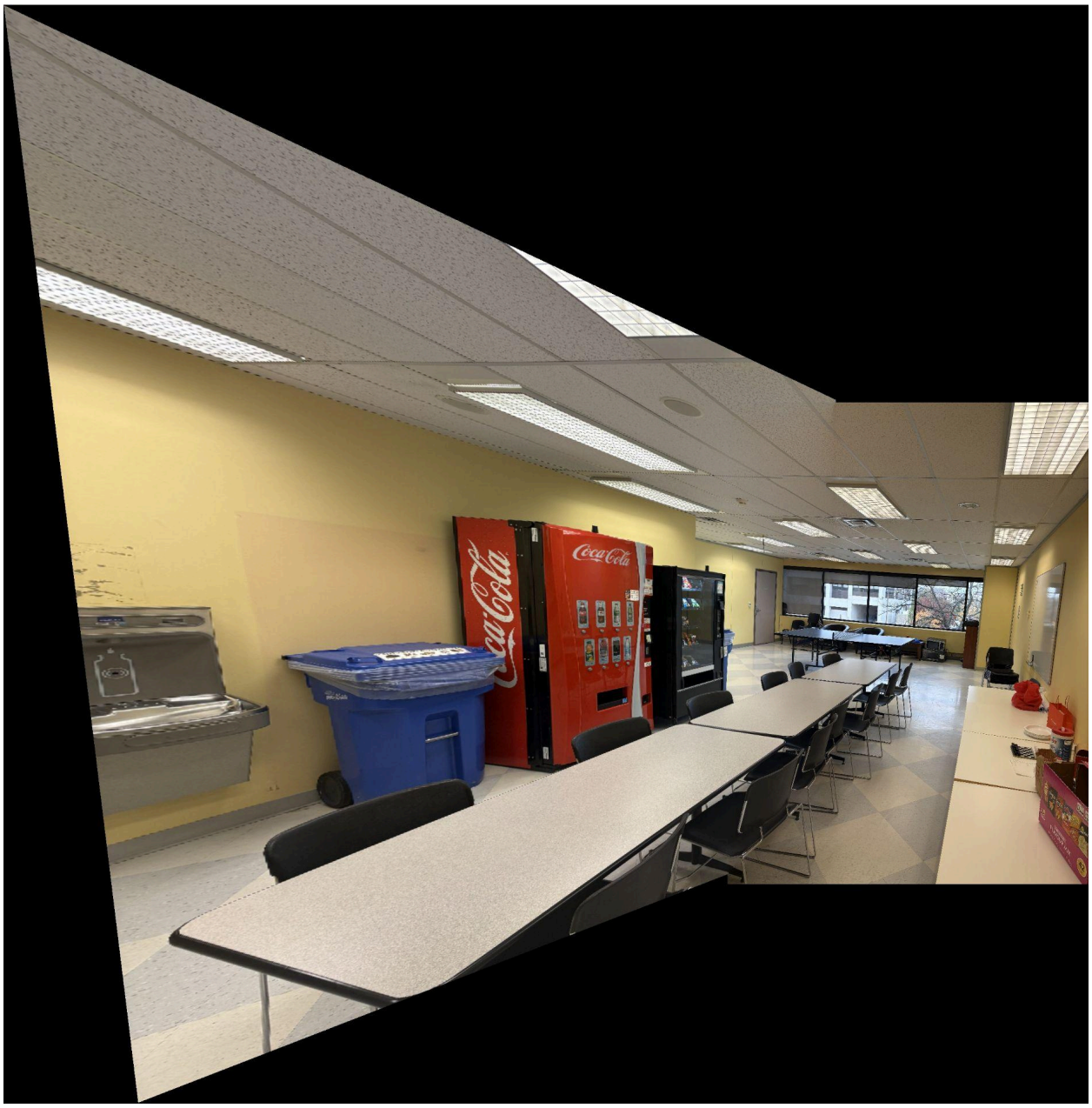
Auto mosaic（images 2 & 3）: Bilinear + Feather Blend



Auto mosaic（images 2 & 3）: Nearest Neighbor + Laplacian Blend

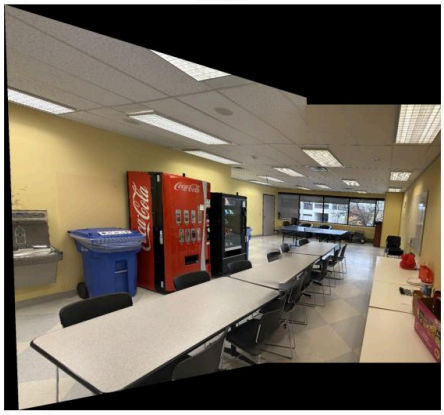


Auto mosaic（images 2 & 3）: Bilinear + Laplacian Blend

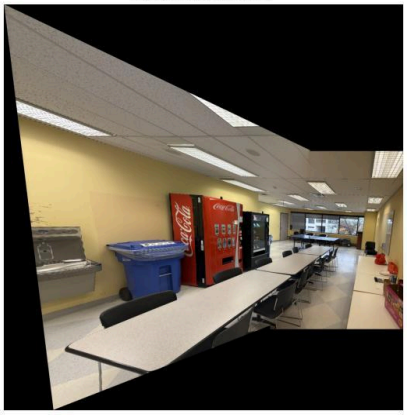


Manual vs Automatic Mosaic (images 2 & 3), Bilinear + Laplacian Blend

Manual



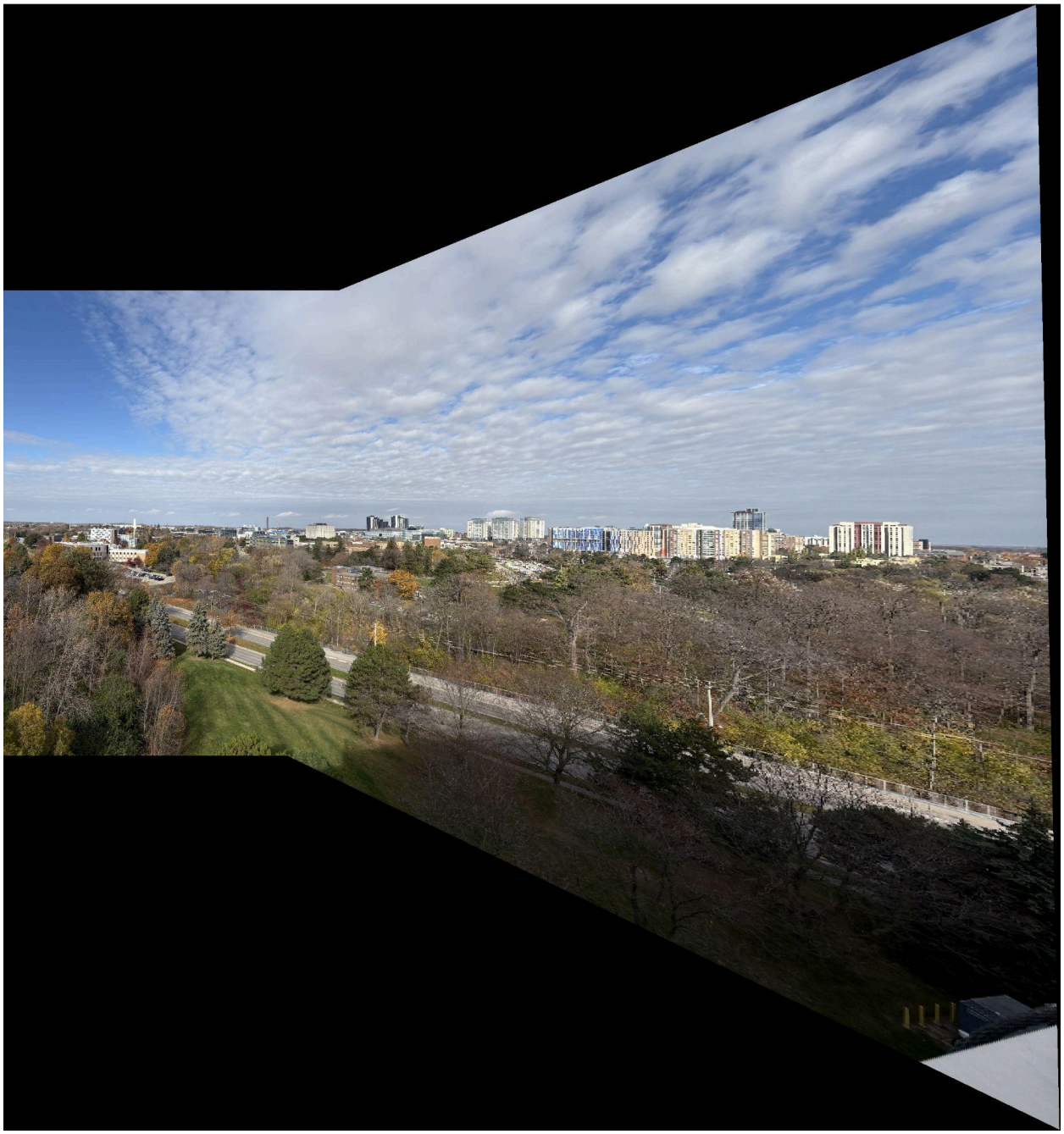
Automatic (RANSAC)



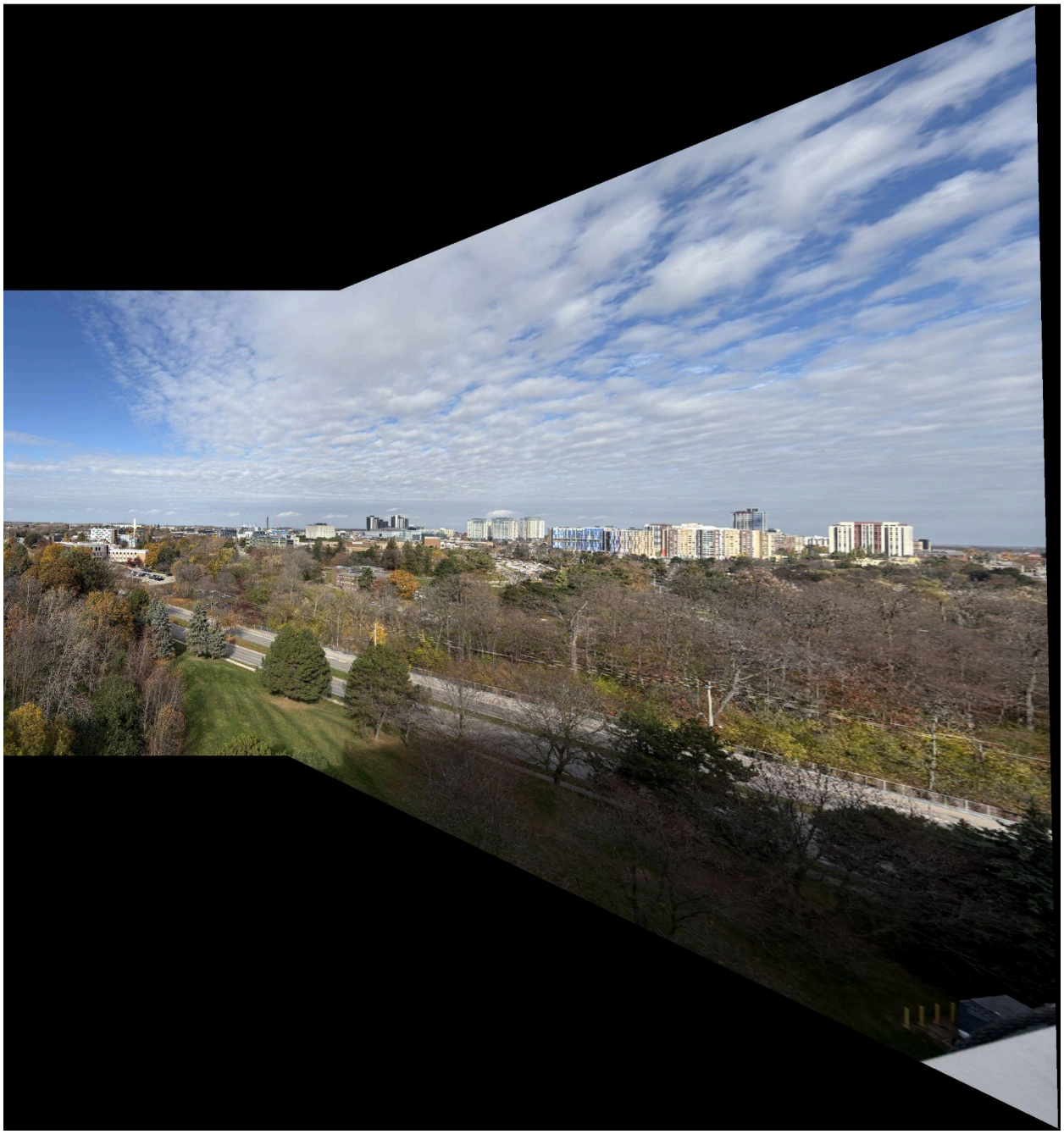
13 matches



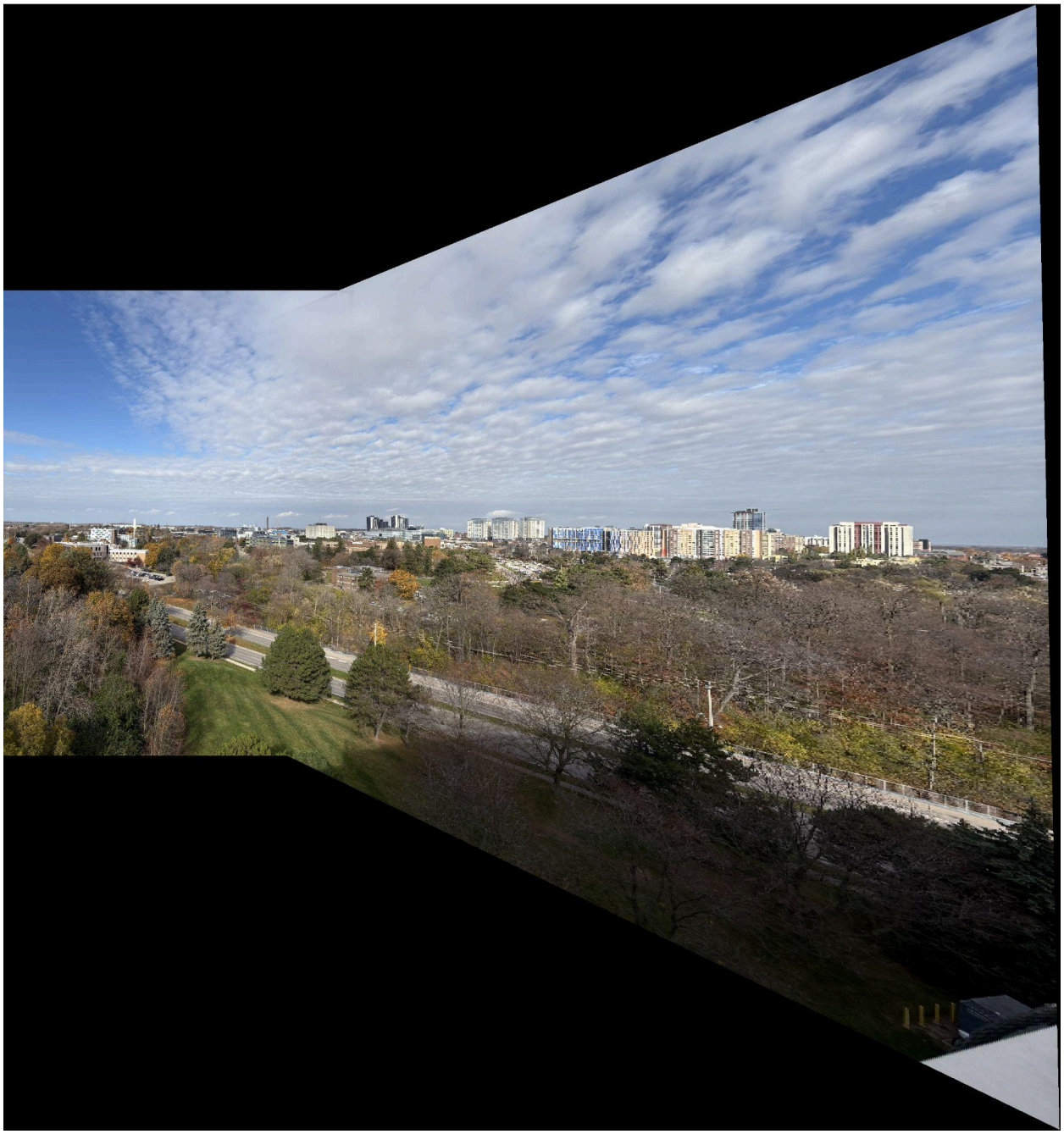
Auto mosaic（images 4 & 5）: Nearest Neighbor + Feather Blend



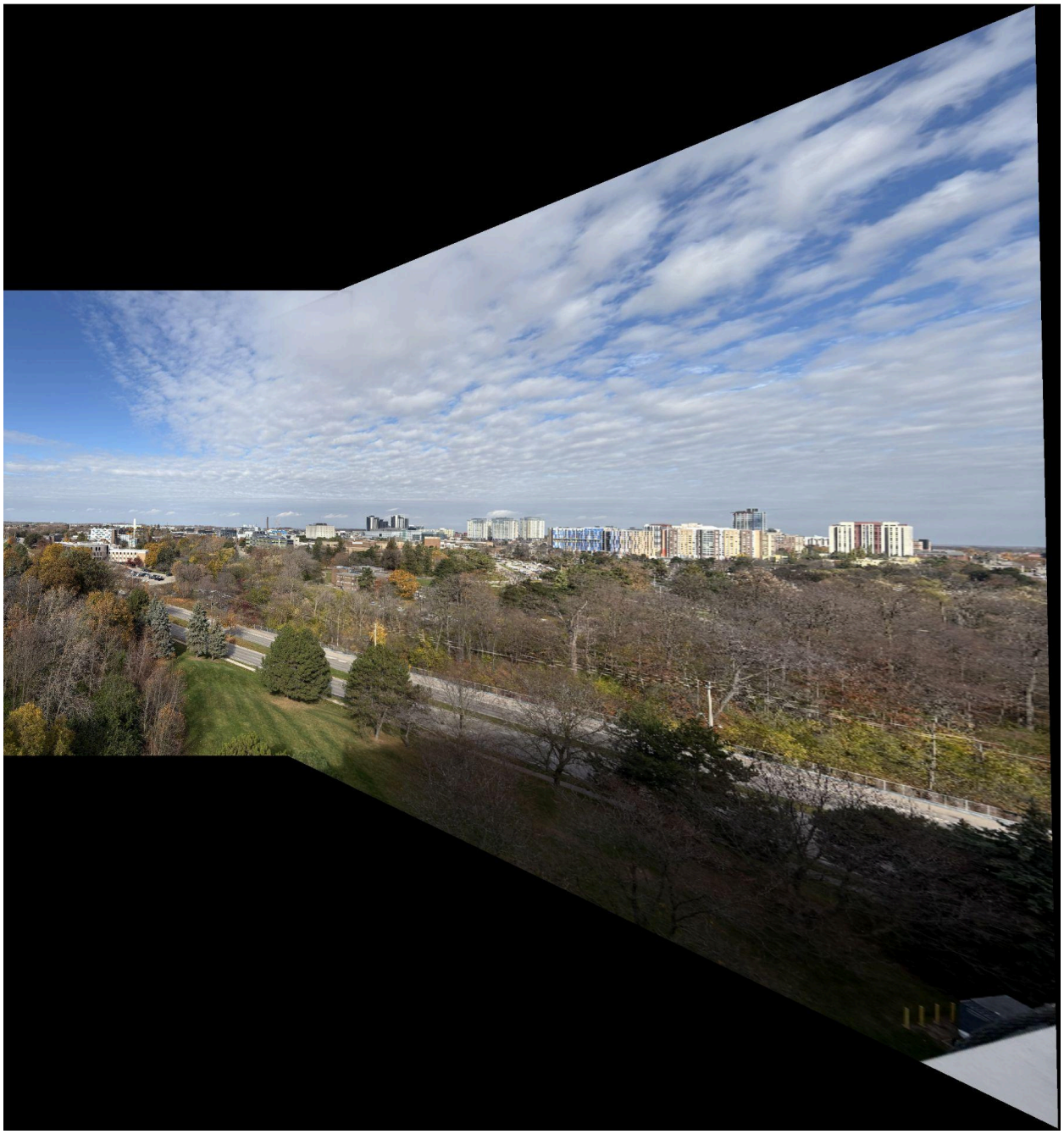
Auto mosaic（images 4 & 5）: Bilinear + Feather Blend



Auto mosaic（images 4 & 5）: Nearest Neighbor + Laplacian Blend



Auto mosaic（images 4 & 5）: Bilinear + Laplacian Blend





RANSAC improved the feature matching drastically by removing the invalid matched features. Overall, the automatic mosaic creates better results as it removes human error from the key point selection. This is most visible in the meeting room mosaic, where the manual version shows a slight seam through the TV, while the automatic version lines up cleanly. The homography is also estimated from many verified inliers (13–20 per pair) rather than a few hand-clicked points, so small errors in any single point have much less effect on the final alignment.

What I Learned:
I learned a lot about image stitching and the process of creating a homography matrix. The most interesting thing part of the assignment was the rectification as it showed how to crop a rectangular area out of an image. All you need is a homography matisse between the area being cropped and a blank area the same size. Creating the mosaics was very interesting as they resemble the creation of a panoramic picture. Part A was very beneficial to understanding the intricacies of how the process works, part B helped understand how to automate the process to improve the results and speed up the process.